

2026



MATHÉMATIQUES ET INFORMATIQUE

Sciences

Université Paris Cité

Cahier de conception général et détaillé

**Développement d'une application de
déquantification**



*Développement sous la
direction du groupe L2F2*

Direction : David JANISKEK

CAHIER DE CONCEPTION

Général et détaillé

Projet L2F2 – Déquantification

Identification du document		Eléments de vérification du document	
Référence	CGD-L2F2	Validé par :	<i>Gaël MAHE</i>
Date	19/04/26	Validé le :	19/04/26
Auteurs	<i>KANGNI Dina, ZOUAD Mohammed, ACHAK Salim et BENALI Abdallah</i>	Soumis le :	19/04/26
Version du document		Confidentialité :	Projet académique, réservé aux étudiants UFR Maths-Info de l'université Paris Descartes

Elément d'authentification

	Maitrise d'ouvrage	<i>Gaël MAHE et David JANISZEK</i>	
	Maitrise d'œuvre	<i>Equipe L2F2</i>	
	Chef de projet	Abdallah BENALI	

Mots clés : *quantification, déquantification, jeu de données*

SOMMAIRE

1. Introduction	5
1.1. Objectifs	5
1.2. Contexte	5
2. Guide de lecture	6
2.1. Maitrise d'œuvre	6
2.2. Maitrise d'ouvrage	6
3. CONCEPTION GENERALE DU PRODUIT	6
3.1. Décomposition fonctionnelle	6
3.2. Architecture générale	8
3.3. Dépendance externe	9
4. CONCEPTION DETAILLE DU PRODUIT	11
a. <i>arbre.jl</i>	11
b. <i>Construction.jl</i>	15
c. <i>dequantification.jl</i>	19
d. <i>Interface.jl</i>	26
e. <i>L2F2_Dequantification_App.jl</i>	47
5. Planification	48
6. Annexe	49
Figure 2 : Diagramme d'activité de l'application L2F2	49
Figure 3 : Architecture complète du projet L2F2	50
7. Glossaire	52
8. Reference	52
9. Index	53

1. Introduction

Ce cahier de conception décrit l'architecture générale et détaillée du projet universitaire L2F qui sera développé par la deuxième équipe de développement composée de ACHAK Salim, BENALI Abdallah et KANGNI Dina et ZOUAD Mohammed.

KANGNI Dina, ZOUAD Mohammed, ACHAK Salim et BENALI Abdallah l'ont rédigé sous la direction de Gaël MAHE qui en assure la relecture finale et l'approuve.

1.1. Objectifs

Ce document décrit l'architecture ainsi que la conception détaillée de l'application de déquantification du groupe L2F2. Il décrit les interactions, l'architecture et plus précisément chaque module et fonction conformément aux besoins exprimés dans le cahier des charges et dans le cahier de recette.

Il est la base de la programmation et a pour but de permettre aux développeurs de le traduire et de rendre compte des exigences de la maîtrise d'ouvrage.

1.2. Contexte

Ce projet consiste à développer une application de déquantification.

La quantification consiste à représenter des données continues en données discrètes. Notre projet s'intéresse plutôt à la sous-quantification à 1 bit.

La sous-quantification à 1 bit est un type de quantification qui vise à quantifier des données qui ont déjà été quantifiées une fois. Pour être plus précis, elle supprime le bit de poids faible.

On ne peut pas retrouver les données originales après cette opération.

Le but de l'équipe L2F2 est donc d'implémenter un algorithme qui permettrait de retrouver les données originales ou un échantillon de données contenant l'original.

Pour ce faire, il pourra générer et exploiter l'histogramme P du jeu de données x entré par l'utilisateur, et générer aussi sa version sous-quantifiée à 1 bit qu'on appellera xQ .

Ces données générées permettront de construire et d'élaguer un arbre des possibles qui retournera un nombre réduit de données trouvées. x devra y figurer.

2. Guide de lecture

Ce projet est composé de deux acteurs que nous allons détailler et qui pourront utiliser ce document selon les modalités énoncées.

2.1. Maitrise d'œuvre

La maitrise d'œuvre est assurée par la deuxième équipe du projet L2F. Elle est composée de quatre membres : KANGNI Dina, BENALI Abdallah, ACHAK Salim et ZOUAD Mohammed.

Elle utilise ce document pour avoir une vue d'ensemble du projet mais aussi identifier les modules et les dépendances.

Ce document est conçu pour être facilement traduisible pour la phase de développement de l'application et rendre compte le plus fidèlement possible des besoins de la maitrise d'ouvrage.

2.2. Maitrise d'ouvrage

La maitrise d'ouvrage est représentée par l'encadrant du projet, Gaël MAHE, ainsi que du responsable de l'UE projet, David JANISZEK.

Elle utilise ce document pour vérifier la conformité technique de l'application avec le cahier des charges et le cahier de recette. Ce document lui permettra aussi de vérifier que les livrables seront conformes et que les objectifs sont atteints.

3. CONCEPTION GENERALE DU PRODUIT

3.1. Décomposition fonctionnelle

Cette application est développée en *Julia* (version 1.10).

Conçu en 2012, il permet de faire des calculs scientifiques et de la modélisation numérique, ce qui nous intéresse. On pourra se référer tout au long du développement, à la documentation disponible sur le [site officiel de Julia](#).

Il est important de se rappeler que la spécificité de Julia est sa rapidité de calcul comparable à celle du langage C. Cependant, il est nécessaire de demeurer patient puisque le langage reste lent lorsqu'il s'agit de compiler ou d'installer les dépendances pour la première fois.

L'application n'utilisera aucune logique d'orienté objet. Seules des structures seront implémentées.

Le développement ainsi que les livrables doivent être compatibles avec Linux, Windows et MacOS. Une mémoire centrale de minimum 4 Go est requise ainsi qu'une carte graphique compatible avec OpenGL 3.3+.

Nous pouvons repérer quatre grands domaines d'activité qui structurent l'application :

- **Préparation des données**

Il s'agit de la première responsabilité que l'application devra endosser.

Ce domaine regroupe toutes les opérations nécessaires à faire avant l'exécution de l'algorithme de déquantification, comme la lecture du fichier binaire x soumis par l'utilisateur, et en interne la génération de xQ , la version sous-quantifiée à 1 bit de x , et son histogramme P .

- **Algorithme de déquantification**

Il s'agit du cœur de l'application. L'application devra pouvoir construire et surtout élaguer un arbre représentant la séquence des x possibles.

- **Interface graphique**

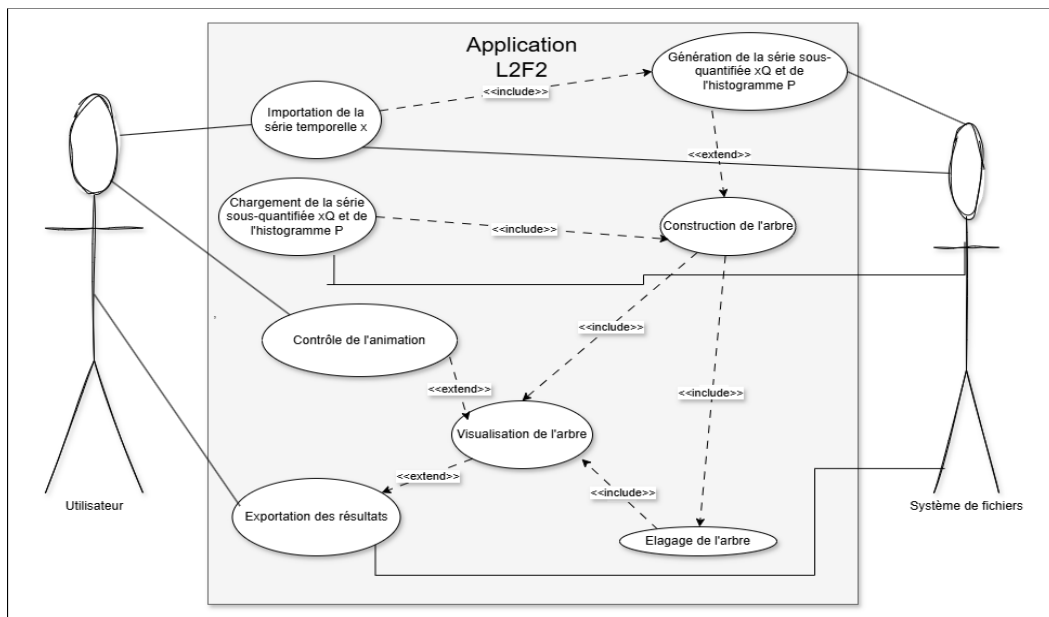
L'utilisateur sera en contact direct avec des éléments afin de pouvoir visualiser l'algorithme. Ces éléments regroupent les boutons de navigation, la visualisation de l'arbre et les messages d'erreurs.

- **Gestion des fichiers**

En complément de la préparation de données, il est nécessaire que notre application puisse naviguer de manière claire dans le système de fichiers en assurant le bon emplacement des fichiers qui seront générés et exportés.

Notre application fait intervenir deux acteurs externes qui communiquent avec le système. D'un côté l'utilisateur qui interagit directement avec l'interface graphique, importe les données, lance l'algorithme, visualise l'animation et peut exporter les résultats. De l'autre côté nous avons *l'explorateur* de fichiers qui fournit les données à importer, reçoit les fichiers exportés et stock xQ et P . (cf. figure 1)

Figure 1 : diagramme UML d'utilisation de l'application



A l'ouverture de l'application, l'utilisateur peut commencer par importer ses données depuis son explorateur de fichiers. Une fois les données chargées, il lance l'algorithme et interagit avec l'animation via des boutons de navigation (arrêter ou lancer). Tout au long de la construction et de l'élagage de l'arbre, un nombre représentant le nombre de branches construites sera affiché et mis à jour dans l'interface graphique. L'utilisateur a la possibilité de récupérer les résultats générés à la fin de l'animation. Ces fichiers seront récupérés dans une archive Zip.

3.2. Architecture générale

L'architecture de notre application suit le plan de la *figure 3* détaillée dans la section *Annexe*.

La logique de notre application peut se décomposer en deux programmes distincts :

Nous avons d'une part le **programme auxiliaire** qui se charge du domaine de la préparation des données. Ce programme prend en entrée x et génère xQ et P . Il s'appuie sur *construction.jl* qui génère xQ , la série sous quantifiée dans un format binaire et la structure de l'histogramme P . Ce dernier est représenté sous la forme d'un dictionnaire.

La partie principale de l'application sera faite dans le **programme principal**. Il est composé de *dequantification.jl* qui prend en entrée xQ et P . Il génère toutes les solutions trouvées à partir de l'élagage de l'arbre.

L'ensemble de la GUI est implémenté dans *interface.jl* et *application.jl*. Ce dernier module sert de point d'entrée de l'application.

L'utilisateur installe dans un premier temps les dépendances via des scripts de lancement propre à chaque système d'exploitation (*install.bat* pour Windows et *install.sh* pour Linux).

Ces scripts installent automatiquement les dépendances via les fichiers de configuration *Project.toml* et *Manifest.toml*.

Par la suite, à chaque fois qu'il voudra lancer l'application il n'aura plus qu'à exécuter un script de lancement (*run.bat* pour Windows et *run.sh* pour Linux).

Le livrable doit contenir de la documentation incluant le cahier de recette, le cahier des charges, le cahier de conception, le plan de tests ainsi que le README et la licence.

Les fichiers générés seront stockés dans le dossier *data*. Mais on prendra soin à chaque lancement de l'application de le vider pour commencer une nouvelle session complètement vierge.

Mis à part l'interface, tous les fichiers sources ont un fichier de test unitaire associé.

Afin de réaliser nos tests, des jeux de données de taille variable sont mis à disposition. Nous les stockons dans le dossier *test/data/input*. Les fichiers générés (xQ et P) seront, sauvegardés dans *src/data/*. Toutes les solutions seront stockées dans le dossier *src/data/temp* qui sera vidé à chaque lancement de l'application.

L'ensemble des tests unitaires est intégré dans *runtests.jl* qui orchestre leur exécution.

3.3. Dépendance externe

Toutes les dépendances externes seront utilisées par *interface.jl*.

Nous utilisons [GLMakie](#), une bibliothèque qui s'appuie sur OpenGL pour assurer le backend graphique. Elle nous permet d'implémenter les boutons d'interactions. Notons alors qu'une carte graphique compatible avec OpenGL 3.3+ est nécessaire.

Cette dépendance présente quelques défauts de compilation avec les versions plus récente de Julia, on conseille donc d'utiliser Julia 1.10.11.

Nous utilisons [GraphMakie](#), une bibliothèque de construction de graphe et qui repose sur GLMakie pour la visualisation de l'arbre.

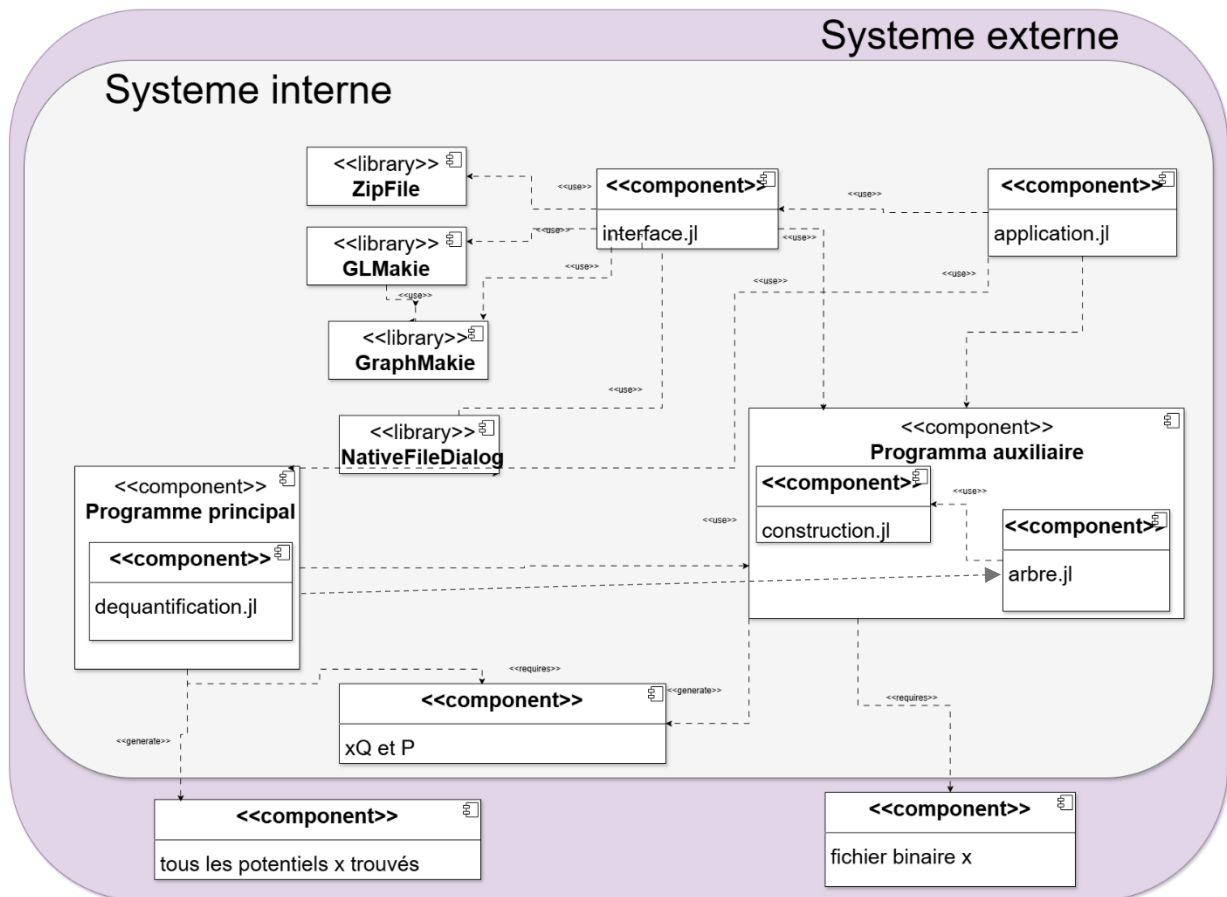
L'ouverture du sélecteur de fichiers natif s'appuie sur [NativeFileDialog](#).

Nous utilisons enfin [ZipFile](#) pour regrouper les résultats dans une archive ZIP téléchargeable si l'utilisateur souhaite les récupérer.

Les tests utilisent uniquement *Test*, un module standard de Julia.

Tous les autres modules n'incluent aucune autre dépendance externe.

Figure 4 : Diagramme de composant du projet L2F



4. CONCEPTION DETAILLE DU PRODUIT

Chaque module possède une fiche d'identité qui décrit son rôle, ses entrées et sorties, et ses dépendances internes et externes.

On détaillera les structures qu'elle contient ainsi que ses fonctions.

Pour chacune de ces dernières, on précisera sa signature, le paradigme utilisé et enfin son algorithme.

Note : dans toute l'application, les valeurs sont codées sur 16 bits signés (*INT16*).

a. arbre.jl

Rôles	Entrée(s)	Sortie(s)	Dépendances internes	Dépendances externes
Contient la structure de l'arbre et les fonctions associés	n/a	n/a	n/a	n/a

Ses structures

- Arbre : contient seulement la racine

```
mutable struct Arbre
    racine :: Union{Nothing, Noeud}
end
```

- Noeud : représente un nœud de l'arbre avec un histogramme associé.

```
mutable struct Noeud
```

```

valeur::Int16
parent::Union{Nothing, Noeud}
pos:: Bool # false = pair (fils bas) true = impair (fils haut)
enfants::Vector{Nœud}

histogramme :: Union{Nothing, Dict{Tuple{Int16, Int16}, Int},
Nothing}

vivant :: Boolean #indique si une branche est vivante ou pas
end

```

Ses fonctions

1. **est_feuille()** – paradigme fonctionnel :

Signature : Nœud -> Bool

Retourne vrai si le nœud n'a pas d'enfant, faux sinon.

```

est_feuille(noeud)
    retourne estVide(nœud.enfants)

```

2. **creer_arbre()** – paradigme fonctionnel

Signature : Nothing -> Arbre

Créer un arbre et l'initialise.

```

creer_arbre()
    racine <- Nœud vide
    retourne Arbre(racine)

```

3. **créer_noeud()** – paradigme fonctionnel

Signature : Int16 x Nœud x Bool x Dict -> Noeud

Prend en entrée la valeur du nœud ainsi que son parent, l'indicateur pos qui vérifie si le nœud est pair ou impair et l'histogramme du parent.

Crée et retourne un nouveau nœud avec un histogramme copié (puisque'on va gérer de manière indépendante l'historgramme de chaque nœud).

```
créer_noeud(valeur, parent, pos, histogramme)  
    retourne Nœud(valeur, parent, pos, Nœud, copie(histogramme))
```

4. ajouter_enfant() – paradigme impératif :

Signature : Arbre x Nœud x Int16 x Bool x Dict{Tuple{Int16, Int16}, Int}} -> Nœud

Prend en entrée l'arbre, le parent du nœud qu'on veut ajouter, son parent, le champs pos et son dictionnaire.

Créer un enfant et l'attache au parent. Retourne le nœud créé.

```
ajouter_enfant(arbre, parent, valeur, position, P)  
    enfant <- Nœud(valeur, histogramme, pos, parent, Nœud[])  
    ajouter enfant à parent.enfants  
    retourne enfant
```

5. supprimer_noeud() – paradigme impératif

Signature : Arbre x Nœud x Nœud -> Nothing

Prend en entrée l'arbre, le parent du nœud qu'on veut supprimer et le nœud.

Supprime un nœud.

```
supprimer_noeud(arbre, parent, nœud)  
    retirer nœud de parent.enfants
```

6. extraire_serie() – paradigme impératif

Signature : Nœud -> Vector

Prend en entrée la branche (= la feuille) qu'on souhaite extraire, la parcourt et la retourne.

```

extraire_serie(noeud)

    sequence <- []
    cur <- n
    tant que cur.parent != null
        ajouter n.valeur à sequence
        cur <- cur.parent
    FinTantQue
    renverser(sequence)

retourne sequence

```

7. **compter_branches()** – paradigme impératif

Signature : *Noeud* -> *Int*

Retourne le nombre de branche du sous arbre du nœud passé en paramètre.

```

compter_branches(noeud)

    si est_feuille(noeud)
        retourne nœud
    finSi

    total <- 0

    pour chaque enfant de nœud
        total = total +compter_branches(enfant)
    finPour

    retourne total

```

b. Construction.jl

Rôles	Entrée(s)	Sortie(s)	Dépendances internes	Dépendances externes
Point d'entrée de notre programme auxiliaire. Il a pour rôle de générer xQ et P.	x	xQ et P	n/a	n/a

Ses fonctions

1. lire_serie () – paradigme impératif

Signature : String -> Vector{Int16}

Prend en entrée le chemin de x et stock les données dans un vecteur. Vérifie si le fichier est vide.

```
lire_serie(chemin)

    si le fichier n'existe pas
        #on lève une erreur
        erreur("Fichier introuvable")

    finSi

    #sans cette réinterprétation, on lit les valeurs comme des UInt8
    au lieu de les lire comme des Int16

    vecteur = reinterpret(Int16, lire(chemin))

    si estVide(vecteur)
        #on lève une erreur
```

```
    erreur("Fichier vide")  
  finSi
```

#la fonction collect() nous permet de récupérer vecteur sous la forme d'un vecteur

```
retourne collect(vecteur)
```

2. sous_quantifier() – paradigme fonctionnel

Signature : Vector{Int16} -> Vector{Int16}

Prend en entrée la série temporelle de x construit avec lire_serie() et retourne xQ dans le même format. Cette fonction va forcer le bit de poids faible de la valeur courante à 0 afin de s'assurer de la parité. L'opération valeur & (-2) met ce bit à 0.

```
fonction sous_quantifier(x)  
  resultat = []  
  pour chaque valeur dans longueur(x)  
    resultat[valeur] = x[valeur] ET (-2)  
  finPour  
  retourne resultat
```

3. construire_p() – paradigme impératif

Signature : Vector{Int16} -> Dict{Tuple{Int16, Int16}, Int}

Prend en entrée le vecteur x.

Retourne le dictionnaire de P avec comme clé le couple x[n-1], x[n], et comme valeur son nombre d'apparition.

Pour chaque clé, on incrémente la valeur.

```
construire_p(x)  
  P <- dictionnaire vide
```



```

pour n de 2 à longueur(x)
    c <- (x[n-1], x[n])
    si c n'existe pas dans P
        P[c] <- 1
    finSi
    sinon
        P[c] <- P[c] + 1
    finSinon
finPour
retourne P

```

4. sauvegarder_xQ() – paradigme impératif

Signature : Vector{Int16} x String -> Nothing

Sauvegarde xQ dans la chaine entrée en paramètre. Cette chaine représente le chemin de xQ.

```

sauvegarder_xQ(xQ, chemin)
    ouvrir chemin en mode écriture
    pour chaque n dans xQ
        écrire n dans le fichier
    finPour

```

5. sauvegarder_P() – paradigme impératif

Signature : Dict{Tuple{Int16, Int16}, Int} x String ->Nothing

Sauvegarde P dans la chaine entrée en paramètre. Cette chaine représente le chemin de P.

```
sauvegarder_P(P, chemin)
```

```
    ouvrir chemin en écriture
```

```
    pour chaque (cle, c) dans P
```

```
        écrire(« cle[1], cle[2], c») dans le fichier
```

```
    finPour
```

6. lire_P() – paradigme impératif

Signature : String -> Dict{Tuple{Int16,Int16},Int}

Prend en entrée le chemin du fichier dans lequel P est enregistré et construit sa représentation dans l'algorithme sous la forme de dictionnaire.

```
lire_P(fichie_p)
```

```
    P <- dictionnaire vide
```

```
    Ouvrir fichier p
```

```
    Tant qu'on n'est pas arrivé à la fin du fichier
```

```
        Lire cle1
```

```
        Lire cle2
```

```
        Lire valeur
```

```
        //construction du dictionnaire
```

```
        P[(cle1, cle2)] <- valeur
```

```
    retourne P
```

c. dequantification.jl

Rôles	Entrée(s)	Sortie(s)	Dépendances internes	Dépendances externes
Point d'entrée du programme principal. Son rôle est de générer toutes les solutions trouvées.	xQ et P.	Toutes les solutions trouvées.	arbre.jl	n/a

Note : Afin de faciliter le débogage, nous allons mettre des logs.

Nous définissons donc la constante `DEBUG` en début du fichier:

```
const DEBUG = {true en session de débogage sinon,  
               false
```

Ses fonctions

1. `est_compatible()` – paradigme fonctionnel

Signature : `Noeud × Int16 → Bool`

Vérifie si le couple `Noeud` et `Int16` est compatible par rapport à `P`.

```
est_compatible(nœud, valeur)  
    si (noeud.valeur, valeur) dans noeud.histogramme  
        retourne noeud.histogramme[(noeud.valeur, valeur)] > 0  
    finSi  
    DEBUG && affiche("Histogramme non équivalent avec le noeud")  
  
    retourne faux
```

2. elaguer() – paradigme récursif

Signature : Arbre × Noeud{Int16} → Nothing

Supprime une branche de l'arbre en supprimant une feuille et en remontant récursivement si le parent devient feuille à son tour.

```
elaguer !(arbre, n)
  si n.parent = null
    retourne
  finSi
  n.vivant <- faux

  si n.parent.parent != nothing ET aucun enfant vivant
    elaguer!(arbre, n.parent)
  finSi
```

3. sauvegarder_solution() – paradigme impératif

Signature : Vector{Int16} × String × Ref{Int} → String

Cette fonction prend en entrée une solution trouvée grâce à l'élagage de l'arbre, le chemin du dossier de sauvegarde et le compteur de solutions. Elle construit le nom du fichier, écrit la série dans un fichier binaire et retourne son chemin.

```
sauvegarder_solution(solution, chemin_dossier, compteur_solutions)
  compteur_solutions <- compteur_solutions + 1
  numero <- conversion compteur_solutions en String
  chemin_fichier <- chemin_dossier + "/solution_" +
compteur_solutions + ".dat"
  ouvrir chemin_fichier en mode écriture
  écrire solution dans le fichier

retourne chemin_fichier
```

4. developper() – paradigme récursif

Signature : Arbre x Nœud x Vector{Int16} x Int x Function x Function x Function x String x Ref{Int} x Ref{Bool}-> Nothing

Fonction centrale qui développe et élague l'arbre binaire des solutions.

Elle prend en entrée :

- l'arbre pour la mise à jour de l'interface graphique,
- le nœud courant,
- la série sous-quantifiée xQ,
- le niveau courant dans xQ,
- les fonctions *mis_a_jour_arbre()*, *mis_a_jour_branches()*, *ajouter_solution()* du module *interface.jl*,
- le dossier de sauvegarde des solutions trouvées,
- le compteur de solutions initialisé dans *dequantifier()* qui va nous permettre de nommer les fichiers (solutionX_1, solutionX_2, ..., solutionX_n).
- la référence *est_lance* qui permet à l'interface d'interrompre l'algorithme

Notons que nous passons le compteur de solutions et le dernier paramètre permettant d'interrompre l'algorithme en tant que référence (Ref{Int}) pour pouvoir les partager entre les appels récursifs.

Déroulé de la fonction

L'algorithme parcourt l'arbre en profondeur. Au nième niveau, le nœud courant a deux fils possibles au niveau n+1 : la valeur paire (soit xQ(n)) et la valeur impaire (soit xQ(n)+1).

Pour chacun de ses fils, on vérifie que le couple (x(n),x(n+1)) possède une occurrence dans l'histogramme du père.

Si c'est le cas, le fils hérite de cet histogramme et décrémente le couple qui maintenant existe dans l'arbre.

On applique cet algorithme récursivement au niveau n+2.

Dans un but d'optimisation de la mémoire, on supprime l'histogramme du père, une fois ses enfants possibles créés puisqu'il peut être retrouvable dans ses fils.

Si aucun des deux fils n'est possible, le père devient donc une feuille et on l'élague. Si son frère est aussi une feuille, leur père est supprimé à son tour, et ainsi de suite, récursivement.

Lorsqu'une série est trouvée, on l'extrait via *extraire_solution()*. Cette situation fera office de cas de base de cette fonction récursive.

A chaque modification de l'arbre, l'interface est mise à jour via la fonction *mis_a_jour_arbre()* du module *interface.jl*.

```
develloper(arbre, nœud, xQ, niveau, mis_a_jour_arbre,
mis_a_jour_branches, ajouter_solution, chemin, compteur_solutions,
est_lance)

//Arret de l'algorithme si l'utilisateur a cliqué sur le bouton
arrêter

    si !est_lance
        retourne rien
    finSi

// cas de base : on a parcouru tout xQ donc cette branche est une
solution

    si niveau = longueur(xQ)+1
        DEBUG ET affiche("[SOLUTION TROUVEE AU NIVEAU ", niveau,
        "]")

        solution <- extraire_serie(noeud)
        chemin_solution <- sauvegarder_solution(solution, chemin,
compteur_solutions)
        ajouter_solution(chemin_solution)
    finSi
    sinon
        xQn <- xQ[niveau]
        valeurs_possibles <- [xQn, xQn+Int16(1)]
        valide <- 0

        DEBUG ET affiche("Initialisation : fils_pair=", xQ[1], "
fils_impair=", xQ[1] + Int16(1))
```

```

DEBUG ET affiche("Arbre créé, début de la recherche")

pour chaque valeur dans valeurs_possibles
    si !est_lance
        retourne rien
    finSi

    si estCompatible(noeud, valeur)
        P_copie <- copie(noeud.histogramme)
        P_copie[(noeud.valeur, valeur)] <-
P_copie[(noeud.valeur, valeur)] - 1
        enfant <- ajouter_enfant!(arbre, noeud, valeur,
valeur % 2 != 0, P_copie)

        valide <- valide + 1

        developper(arbre, enfant, xQ, niveau + 1,
mis_a_jour_arbre(), mis_a_jour_branches (), ajouter_solution(),
chemin, compteur_solutions, est_lance)
    finSi
finPour

//On supprime l'histogramme du père
noeud.histogramme <- null

//Si valide = 0, cela signifie que aucun des deux enfants est
compatible avec l'algorithme.

Leur parent doit aussi être élagué de manière récursive

```

```
Si le noeud n'a pas d'enfants alors que nous ne sommes pas
au dernier niveau, il faut aussi élaguer//
```

```
si valide = 0 OU estVide(nœud.enfants)
    si niveau <= longueur(xQ)
        DEBUG ET affiche("ELAGAGE noeud=", noeud.valeur, "
parent=", noeud.parent.valeur)
        elaguer!(arbre, nœud)
        mis_a_jour_branches(compter_branches(arbre.racine))
    finSi
mis_a_jour_arbre(arbre)
finSinon

mis_a_jour_arbre(arbre)

retourne rien
```

5. dequantifier() – paradigme impératif

Signature : Vector{Int16} × Dict{Tuple{Int16, Int16}, Int} × Function x Function x Function x String x Ref{Bool} → Nothing

Cette fonction est le point d'entrée de notre programme principal.

Elle prend en entrée :

- xQ,
- l'histogramme original,
- les fonctions *mis_a_jour_arbre()*, *mis_a_jour_branches()* et *ajouter_solution()* du module *interface.jl*,
- le chemin du dossier de sauvegarde des solutions,
- la référence *est_lance* qui permet d'interrompre l'algorithme.

Elle initialise l'arbre et copie l'histogramme, pour garantir l'existence de l'original.

Elle crée ensuite les deux premiers nœuds (xQ(1) et xQ(1)+1).

Elle initialise le compteur de solutions pour nommer les fichiers générés.

Puis elle appelle *developper()* sur chacun des deux premiers nœuds.

```
dequantifier(xQ, P, mis_a_jour_arbre, mis_a_jour_branches,
ajouter_solution, chemin, est_lance)

DEBUG ET affiche("Appel de dequantifier")
DEBUG ET affiche("Taille de xQ : ", length(xQ))

arbre <- creer_arbre()
DEBUG ET affiche("xQ[1] = ", xQ[1], " enfant_gauche = ", xQ[1], "
enfant_droit = ", xQ[1] + Int16(1))
DEBUG ET affiche("arbre cree")
compteur_solutions <- 0
arbre.racine <- Nœud(0, null, false, Nœud, null)
enfant_gauche<- ajouter_enfant!(arbre, arbre.racine, xQ[1], false,
copie(P))

enfant_droit <- ajouter_enfant!(arbre, arbre.racine, xQ[1] +
Int16(1), true, copie(P))

mis_a_jour_arbre(arbre)

developper(arbre, enfant_gauche, xQ, 2, mis_a_jour_arbre(),
mis_a_jour_branches(),ajouter_solution(), chemin,
compteur_solutions, est_lance)
//Si l'utilisateur appuie sur arrêter, on stoppe l'algorithme
si !est_lance
    retourne rien
finSi
```

```

developper(arbre, enfant_droit, xQ, 2, mis_a_jour_arbre(),
mis_a_jour_branches(), ajouter_solution(), chemin,
compteur_solutions, est_lance)

mis_a_jour_branches(compter_branches(arbre.racine))

DEBUG ET affiche("==Fin de déquantifier")
DEBUG ET affiche("Taille de xQ : ", length(xQ))

```

d. Interface.jl

Rôles	Entrée(s)	Sortie(s)	Dépendances internes	Dépendances externes
Gère l'interface graphique de l'application (visualisation de l'arbre et interaction avec l'utilisateur à travers les boutons)	n/a	n/a	Dequantification.jl Arbre.jl	GLMakie, GraphMakie, NativeFileDialog, ZipFile

Note : Afin de faciliter le débogage, nous allons mettre des logs.

Nous définissons donc la constante `DEBUG_INTERFACE` en début du fichier:

```
const DEBUG_INTERFACE = {true en session de débogage sinon,  
                          false
```

Afin de garantir que le programme trouve toujours le dossier data dans lequel on va sauvegarder xQ, P et les solutions, nous définissons aussi en début de fichier la constante DOSSIER_DATA qui construit à l'aide de la fonction de base *dirname(@__FILE__)* son chemin absolu.

```
const DOSSIER_DATA = joinpath(dirname(@__FILE__), "data")
```

De plus, pour des raisons de portabilité, nous utiliserons la fonction *joinpath()* une fonction de base de *Julia* afin de construire les chemins des fichiers et des dossiers.

1. Créer_interface() – paradigme impératif

Signature : Nothing -> Nothing

Point d'entrée de l'interface.

Initialise l'interface graphique. Pour ce faire elle commence par activer GLMakie et s'assure que le dossier de stockage des solutions est vide.

Après avoir configuré l'interface graphique elle configure les observables et les widgets. Elle finit par dessiner l'arbre et gérer les actions de chacun des boutons.

L'interface graphique est illustrée par la figure 4 (voir Annexe).

```
créer_interface()
```

```
#INITIALISATION DE LA FENETRE
```

```
#initialisation de GLMakie
```

```
GLMakie.activate!(title = "Projet de  
déquantification", focus_on_show = true)
```

```
#creation de la figure principale
```

```
figure = Figure(size = (1200, 500), figure_padding = 60,  
backgroundcolor = :lightblue)
```

```

#réinitialisation du dossier temp qui contiendra les solutions
trouvées par l'algorithme :

#creation du dossier au cas ou il n'existerait pas
mkpath(joinpath(DOSSIER_DATA, "temp"))

#parcourt de la liste des fichiers dans le dossier data
pour f dans readdir(joinpath(DOSSIER_DATA, "temp"), join=true)
    supprime f
finPour

#OBSERVABLES

#Observable du graphe: contient le graphe et permet de le mettre
à jour
graphe_obs          = Observable(SimpleDiGraph())

#Observable de la position des noeuds
positions_obs = Observable(Point2f[])

# Observable du nombre de branches restants
branches_obs      = Observable(0)

#changement dynamique du texte affiché à l'écran
texte_dynamique = @lift(string($branches_obs) * " branche(s)")

#Observable des solutions restantes.
solution_obs      = Observable(String[])

```

```

#Observable indiquant si l'animation est en cours. Vaut true si
elle l'est, false sinon

est_lance          = Observable(false)

# Ref{Bool} passé à l'algorithme pour l'interrompre
continuer = Ref{Bool}(true)

#texte d'erreur affiché dans l'axe
texte_erreur = Ref{Any}(nothing)

#texte de succès affiché à l'ecran
message_succes_obs = Observable("")

#couleur des nœuds
Couleur_obs = Observable(Vector{Symbol}())

////

Références indiquant si les données (xQ et P) ont bien été
chargées
////

xQ_charge = Ref{Union{Nothing, Vector{Int16}}}(nothing)
P_charge  = Ref{Union{Nothing, Dict{Tuple{Int16,Int16},
Int}}}(nothing)

///

Layout et les boutons

COLONNE GAUCHE

```

contient le bouton "exporter les solutions" ainsi que le bouton télécharger xQ et P

COLONNE DROITE

contient l'arbre ainsi que les boutons de navigation (lancer et arrêter) et les boutons (importerx/xQ et P)

On organise l'espace de l'interface sous forme de grille (GridLayout). On définit ensuite la hauteur des lignes (rowsize) et la largeur des colonnes (colsize) qui la composent. On ajuste aussi l'espacement entre les lignes (rowgap) et entre les colonnes (colgap).

////

#COLONNE GAUCHE

```
layout_gauche = figure[1, 1] = GridLayout(alignmentmode =  
Outside(20))
```

```
colsize!(figure.layout, 1, Fixed(270))
```

#sous-grille pour aligner les boutons

```
layout_boutons_export = layout_gauche[2, 1] = GridLayout()
```

```
bouton_exporter = Button(layout_boutons_export[1, 1], label =  
"Exporter les solutions", height = 40, tellwidth = false)
```

```
bouton_telecharger_xQP = Button(layout_boutons_export[1,2],  
label = "Telecharger xQ et P",height = 40, tellwidth = false)
```

```
colgap!(layout_boutons_export,120)
```

#Les solutions

```
Label(layout_gauche[3, 1], "Liste des solutions :", halign =  
:left, padding = (0, 0, 10, 0), fontsize=18, font=:bold)
```

```

        layout_liste = layout_gauche[4,1] = GridLayout(halign =:left,
tellheight=false)

#Box invisible pour pousser tout le contenu vers le haut
layout_gauche[9, 1] = Box(ffigure, visible = false)
rowsize!(layout_gauche, 9, Relative(1))

#COLONNE DROITE
layout_droite = figure[1, 2] = GridLayout()

#creation de la zone de dessin du graphe avec 'Axis'
ax = Axis(layout_droite[1, 1], title = "Déquantification",
titlesize=30,alignmode = Outside(10)) #zone de dessin
hidedecorations!(ax) # On supprime les axes et les coordonnées

#Les boutons en dessous de l'axis (zone de dessin)
grille_bas = layout_droite[2, 1] = GridLayout(tellheight = true)
#Message de succes
label_succes = Label(grille_bas[1,0], text = message_succes_obs,
font = :bold,fontsize= 16, color = :green)

#bouton d'importation x
bouton_import = Button(grille_bas[1, 1], label = "Importer
x", height = 45, tellwidth = false,font = :bold)

#bouton de lancemenent de l'algorithme
bouton_lancer = Button(grille_bas[1, 2], label = "Lancer",
buttoncolor = :green, height = 45, tellwidth = false,font = :bold)

```

```

#bouton d'arrêt de l'algo

bouton_arreter      = Button(grille_bas[1, 3], label = "Arrêter",
buttoncolor = :tomato, height = 45, tellwidth = false,font = :bold)


#bouton d'importation du dossier contenant xQ et P

bouton_import_xQP = Button(grille_bas[1, 4], label = "Importer
xQ et P", height = 45, tellwidth = false, font = :bold)


# GESTION DE L'ESPACE

rowsize!(layout_droite, 1, Relative(1))
rowsize!(layout_droite, 2, Auto())


# Espace entre les boutons et entre le graphe

colgap!(grille_bas, 10)
rowgap!(layout_droite, 10)


# Dessin de l'arbre

graphplot!(ax, graphe_obs,layout = _ -> positions_obs[],
node_size  = 20,node_color = couleur_obs,edge_color =
:gray,arrow_show = false)


# affichage du nombre de branches affiché à l'écran

text!(ax, 0, 0,
      text = texte_dynamique,
      space = :relative,
      align = (:left, :bottom),
      offset = (15, 15),
      fontsize = 24,

```



```

font = :bold)

//
fonctions passées à dequantifier
Permet de lier l'interface directement à l'algorithme
//

#mis_a_jour_arbre_cb reçoit un nouvel arbre et le met à jour
mis_a_jour_arbre_cb = (arbre) -> mis_a_jour_arbre(graphe_obs,
positions_obs, couleur_obs, arbre)

#mis_a_jour_branches_cb reçoit un nouveau chiffre n et met à
jour le nombre de branche
mis_a_jour_branches_cb = (n) -> begin
  DEBUG_INTERFACE ET affiche("mis_a_jour_branches appelé avec ",
n)
  mis_a_jour_branches(branches_obs, n)

//
EVENEMENTS
On définit le comportement de chaque bouton
//

# bouton importer: charge les fichiers importés et affiche les
messages d'erreur ou de réussite
Quand clic sur bouton_import faire
  x <- importer_fichier(ax, texte_erreur, message_succes_obs)
  si estVide(x)

```

```

        preparer_donnees(x, xQ_charge, P_charge, DOSSIER_DATA)
    finSi
    finQuand

    #bouton importer le dossier xQ et P
    Quand clic sur bouton_importxQP faire
        importer_xQP(ax, xQ_charge, P_charge,
        texte_erreur,message_succes_obs)
    finQuand

    #bouton télécharger xQ et P
    Quand clic sur bouton_telecharger_xQP faire
        telecharger_xQP(DOSSIER_DATA,message_succes_obs)
    finQuand

    # bouton de lancement
    Quand clic sur bouton_lancer
        DEBUG_INTERFACE ET affiche("Lancer cliqué")
        Si xQ_charge[] est non null ET P_charge[] est non null
            continuer[]= vrai
            DEBUG_INTERFACE ET affiche("Taille de P:
            ",length(P_charge[]))
            lancer_animation(ax::Axis,texte_erreur::Ref{Any},xQ_charge,
            P_charge,est_lance,graphe_obs, branches_obs,
            solution_obs,mis_a_jour_arbre_cb,
            mis_a_jour_branches_cb,joinpath(DOSSIER_DATA, "temp"), continuer)
        finSi
    finQuand

    #Cadrage automatique de la vue du graphe pour qu'il reste visible

```

```

Quand graphe_obs faire
    autolimits!(ax)
finQuand

# --- bouton arrêter (est géré grâce à la reference continuer)
Quand clique sur bouton_arreter faire
    arreter_animation(est_lance, continuer)
finQuand

# --- bouton exporter ---
Quand clic sur bouton_exporter faire
    telecharger_solutions(solution_obs[],message_succes_obs)
finQuand

#comportement de la liste des solutions
Quand solution_obs faire chemins
    #On remet la liste des solutions à 0
    Pour chaque élément dans layout_liste), on supprime

    Pour (i, chemins) in enumerate(chemins)
        Label(layout_liste[i, 1], basename(chemins), halign = :left)
    finPour
finQuand

//
    affichage de la fenetre
//

DEBUG_INTERFACE ET affiche("OUVERTURE DE L'APPLICATION")

```

Ouverture(*figure*)

Attendre la fermeture de la fenetre

2. `importer_fichier()` – paradigme impératif

Signature : `Axis -> Union{Nothing, Vector{Int16}}`

Prend en entrée l'axe dans lequel va s'afficher l'arbre afin de gérer les erreurs et retourne `x`.

Ouvre l'explorateur de fichier et charge la série s'il n'y a pas d'erreurs.

```
importer_fichier(ax_graphe)
```

```
chemin <- choix du fichier dans le système de fichier
```

```
si chemin = "" #l'utilisateur n'a rien choisi
```

```
    retourner vecteur vide
```

```
finSi
```

```
ouvrir le fichier en mode lecture
```

```
réinterpréter les octets lus comme des entiers 16 bits
```

```
convertir le résultat en tableau et le stocker dans une variable  
serie
```

```
#Traitement du cas où le fichier est vide
```

```
Si serie est vide
```

```
    afficheErreur(ax_graphe, « Fichier vide. »
```

```
    retourne fichier vide
```

```
finSi
```

```
#Traitement du cas où le fichier ne contient pas assez de  
données
```

```
Si longueur(serie) < 2
```

```

    afficheErreur(ax_graphe, « Fichier contenant moins de deux
valeurs »

    retourne vecteur vide

finSi

retourne serie

```

3. preparer_donnees() – paradigme impératif

Signature: Vector{Int16} x Ref{Union{Nothing, Vector{Int16}}} x Ref{Union{Nothing, Dict{Tuple{Int16,Int16}, Int}}} x String → Nothing

Prend en entrée la série originale x, une référence de xQ et de P pour que *lancerAnimation()* puisse les manipuler, et le chemin du dossier de sauvegarde

La fonction calcule xQ, la série sous-quantifiée à 1 bit de x et P l'histogramme des couples. Elle les sauvegarde dans un dossier pour que l'utilisateur puisse les récupérer et les stocke dans leurs références respectives (xQ_ref et P_ref) pour lancer_animation.

```

Préparer_donnees(x, xQ_ref, P_ref, dossier_sauvegarde)

    xQ <- sousQuantifier(x)

    P <- construireP(x)

    xQ_ref[] <- xQ

    P_ref[] <- P

    Creation de dossier_sauvegarde #mkpath()

    Sauvegarde_Xq(xQ_ref, chemin fichier xQ) #jointpath()

    Sauvegarde_P(P_ref, chemin fichier P)    #joinpath()

```

4. lancer_animation() – paradigme impératif

Signature : Axis x Ref{Any}xRef{Union{Nothing, Vector{Int16}}} x Ref{Union{Nothing, Dict{Tuple{Int16,Int16}, Int}}} x Observable{Bool} x Observable{SimpleDiGraph} x Observable{Int}xObservable{Int} x Observable{Vector{String}}xFunctionxFunctionxString xRef{Bool}-> Nothing

La fonction prend en entrée :

- **l'axe dans lequel vit le graphe,**
- **la référence du texte d'erreur qui s'affichera à la place de l'arbre,**
- **les références vers xQ et P,**
- **l'observable *est_lance* indiquant que l'animation est en cours,**
- **l'observable *graphe_obs* contenant l'arbre et se mettant à jour au fur et à mesure de la déquantification,**
- l'observable *branche_obs* mettant à jour le nombre de branches restantes,
- l'observable *solutions_obs* stockant toutes les solutions trouvées,
- les fonctions *mis_a_jour_arbre*, *mis_a_jour_branches*,
- le dossier de sauvegarde des solutions et,
- la référence *est_lance* qui permet d'arrêter l'algorithme si l'utilisateur appuie sur le bouton « arrêter ».

Elle vérifie que les données sont chargées, réinitialise l'interface graphique, met à jour le dossier de sauvegarde et lance la fonction dequantifier.

```
lancer_animation(  
    ax_graphe,  
    texte_erreur,  
    xQ_charge,  
    P_charge  
    est_lance  
    graphe_obs,  
    branche_obs,  
    solution_obs,  
    mis_a_jour_arbre,  
    mis_a_jour_branches,  
    dossier)
```

```
//
```

```
Initialisation et réinitialisation
```

```
- verifie que les données sont chargées, prépare le dossier de  
sauvegarde et change l'etat est_lance à true
```

```

//
DEBUG ET affiche("lancer_animation appelé")

#Verifie si les données sont chargées
  si xQ_charge = null OU P_charge = null
    retourne rien
  finSi

#reinitialise l'interface
graphe_obs <- SimpleDiGraph() #le graphe correspondant à l'arbre
branche_obs <- 0
solution_obs <- chaine vide

# vide le dossier de stockage des solutions
mkpath(dossier) #chemin du dossier
pour chaque fichier dans la liste des fichiers de dossier
  supprimer le fichier
finPour

est_lance <- vrai

//
  Lance l'algorithme en mode asynchrone pour garantir la mise à
  jour du nombre de branche et du graphe parallèlement.

  A l'issue, remet l'état de est_lance à false et liste toutes les
  solutions trouvées

//
@async
  Essayer

```

```

        ajouter_solution = (chemin_solution) -> begin
        liste = solution_obs[]
        ajouter liste à chemin_solution
        solution_obs <- liste

        DEBUG_INTERFACE ET affiche("Solution ajoutée: ",
basename(chemin_solution))

        dequantifier(
            xQ_charge,
            P_charge,
            mis_a_jour_arbre,
            mis_a_jour_branches,
            ajouter_solution,
            dossier, continuer)
    si erreur e
        DEBUG_INTERFACE ET affiche(afficher erreur e)
    finSi
    dans tous les cas executer
        est_lance <- false
    finEssayer

```

5. arrêter_animation() – paradigme impératif

Signature : Observable{Bool}, Ref{Bool}-> Nothing

Prend en entrée l'observable *est_lance* (signal qui prévient l'interface de l'état de l'animation) et la référence *continuer* (signal qui prévient l'algorithme de l'état de l'animation).

Met en pause l'animation. L'arbre et le nombre de branche affiché à l'écran à ce moment conservent leur dernier état.

arrêter_animation(est_lance)


```
est_lance <- faux  
continuer <- faux
```

6. `mis_a_jour_branches ()` – paradigme impératif

Signature : `Observable{Int} x Int -> Nothing`

Prend en entrée l'Observable représentant le nombre de bacnehs affiché et met à jour sa valeur avec la nouvelle mesure.

Met à jour l'Observable du pourcentage.

```
mis_a_jour_branches(branches_obs, nb_branches)  
  
    branches_obs <- nb_branches  
  
    attendre 0.1 secondes
```

7. `télécharger_solutions()` – paradigme impératif

Signature : `Vector{String} x Observable{String}-> Nothing`

Prend en entrée le vecteur des solutions générées et un observable pour afficher un message de succès.

Télécharge les solutions dans un dossier choisi par l'utilisateur en format ZIP.

```
télécharger_solutions(solutions_select, message_succes_obs)  
  
    dossier_dest <- choisir un dossier de destination  
    si solutions_select est vide OU dossier_dest = ""  
        retourne rien  
    finSi  
  
    chemin_zip = joinpath(dossier_dest, "solutions_trouvees.zip")
```

```

w = ZipFile.Writer(chemin_zip)

  pour chemin dans solutions
    ajouter le chemin dans l'archive Zip
    copier(chemin, chemin dossier_dest)
  finPour

fermer(w)

affiche_succes(message_succes_obs, "Toutes les solutions ont été
exporté avec succès")

DEBUG_INTERFACE ET affiche(« Toutes les solutions exportées dans
un fichier Zip")

```

8. telecharger_xQP() – paradigme impératif

Signature: String x Observable{String}-> Nothing

Prend en entrée le chemin du dossier source où xQ.dat et P.ppm sont générés et un observable pour afficher un message de succès.

Copie xQ.dat et P.bin depuis le dossier de l'application vers un dossier choisi par l'utilisateur.

```

télécharger_xQP(dossier_source::String)::Nothing

  dossier_dest <- selectionner un dossier de telechargement
  si dossier_dest = "" OU dossier_dest = null
    retourne rien
  finSi

  copier(xQ.dat de dossier_source, dossier_dest)
  copier(P.bin de dossier_source, dossier_dest)
  DEBUG_INTERFACE && println("Téléchargement de xq et P")

```

9. affiche_erreur() – paradigme impératif

Signature : Axis x String x Référence -> Nothing

Prend en entrée l'axe dans lequel va s'afficher le message d'erreur, le message et la référence du texte d'erreur.

Affiche un message d'erreur en rouge dans la zone de dessin de l'arbre à la place de ce dernier. (*fichier vide* ou *fichier contenant moins de deux valeurs*).

```
affiche_erreur(ax_graphe, message, texte_erreur)

  si texte_erreur est non null
    effacer texte_erreur de ax_graphe
  finSi

  texte_erreur[] = texte !(ax_graphe, message, position =
(0.5,0.5), align = ( :center, :center), couleur = rouge, taille
police = 20)
```

10. affiche_succes() – paradigme impératif

Signature : Observable{String} x String -> Nothing

Prend en entrée l'observable qui permet d'afficher le message de succès et le message. Affiche le message de succès à l'écran en asynchrone.

```
affiche_succes(message_succes_obs, texte)

  message_succes_obs <- texte

  #disparition du message au bout de 3 secondes

  @async begin
    attendre(3)
    message_succes_obs <- ""
```

11. indexer_noeuds() – paradigme impératif

Signature : SimpleDiGraph x Noeud x Dict{Noeud, Int} -> Nothing

Prend en entrée le graphe, le nœud courant et un dictionnaire.

Parcourt l'arbre en profondeur et ajoute un sommet dans le graphe pour chaque nœud rencontré.

SimpleDiGraph modélise un graphe possédant une relation parent->enfant, mutable et compatible avec GLMakie.

Les sommets sont représentés par des entiers consécutifs strictement positifs, ainsi l'implémentation d'un dictionnaire d'indice est nécessaire pour assurer la correspondance entre ces entiers et les nœuds.

```
indexer_noeuds!(graphe, noeud, indices)  
    ajouter un sommet vide dans graphe  
    indices[noeud] = nombre de sommets actuels dans graphe  
  
    pour chaque enfant de noeud.enfants  
        indexer_noeuds!(graphe, enfant, indices)  
    finPour
```

12. relier_aretes() – paradigme impératif

Signature : SimpleDiGraph x Noeud x Dict{Noeud, Int} -> Nothing

Prend en entrée le graphe, le nœud courant et le dictionnaire nœud-indice sommet.

Parcourt l'arbre et ajoute une arête entre le parent et l'enfant dans le graphe.

```
relier_aretes!(graphe, noeud, indices)  
    pour chaque enfant de noeud.enfants:  
        ajouter arête (indices[noeud] -> indices[enfant]) dans  
graphe  
        relier_aretes!(graphe, enfant, indices)  
    finPour
```

13. calculer_x() – paradigme impératif

Signature : Noeud x Dict{Noeud, Float32} x Ref{Int} -> Nothing

Prend en entrée le nœud courant, sa position qui sera inscrite dans le dictionnaire nœud – position et une référence pour compter le nombre de nœud.

Attribue une position à chaque nœud dans l'ordre du parcours en profondeur (gauche à droite).

```

calculer_x!(noeud, x_pos, compteur)
    si noeud est une feuille
        compteur[] += 1
        x_pos[noeud] = Float32(compteur[])
    FinSi
    Sinon
        pour chaque enfant de noeud.enfants
            calculer_x!(enfant, x_pos, compteur)
        finPour
        x_pos[noeud] = (x_pos[premier enfant] + x_pos[dernier
enfant]) / 2
    finSinon

```

14. calculer_profondeur() – paradigme impératif

Signatrure : Nœud x Dict{Noeud,Int} x Int -> Nothing

Prend en entrée le noeud courant, le dictionnaire noeud – profondeur et la profondeur du nœud dans l'arbre.

Calcule la profondeur de chaque noeud dans l'arbre par récursion. La racine a profondeur 0, ses enfants 1, leurs enfants 2, etc.

```

calculer_profondeur!(noeud, profondeurs, prof)
    profondeurs[noeud] = prof

    pour chaque enfant de noeud.enfants:
        calculer_profondeur!(enfant, profondeurs, prof + 1)
    finPour

```

15. convertir_arbre()– paradigme impératif

Signature : Arbre -> SimpleDiGraph x Vector{Point2f} x Vector{Symbol}

Prend en entrée l'arbre et retourne son graphe compatible, les coordonnées (x, - profondeur) de chaque sommet et couleur de chaque sommet (bleu si vivant, :rouge sinon).

Réalise les quatre étapes de la construction du graphe dans l'ordre :

1. indexer_noeuds! — créer un sommet par noeud et mémoriser son id
2. relier_aretes! — tracer les arêtes parent->enfant
3. calculer_x! — calculer la position horizontale
4. calculer_profondeur! — calculer la position verticale Puis construit les vecteurs positions et couleurs en parcourant la table indices.

convertir_arbre(arbre)

graphe, indices, x_pos, profondeurs, compteur = structures vides

indexer_noeuds!(graphe, arbre.racine, indices)

relier_aretes!(graphe, arbre.racine, indices)

calculer_x!(arbre.racine, x_pos, compteur)

calculer_profondeur!(arbre.racine, profondeurs, 0)

pour chaque (noeud, i) dans indices

positions[i] = Point2f(x_pos[noeud], -profondeurs[noeud])

finPour

pour (noeud, i) dans indices

couleurs[i] = noeud.vivant ? :bleu : :rouge

finPour

retourne graphe, positions, couleurs

16. mis_a_jour_arbre – paradigme impératif

Signature : Observable x Observable x Observable x Arbre -> Nothing

Pont entre l'algorithme et l'interface. Appelle convertir_arbre pour traduire l'arbre courant, puis donne le résultat dans les observables GLMakie.

```
mis_a_jour_arbre(graph_obs, positions_obs, couleur_obs, arbre)
    g, pos, col = convertir_arbre(arbre)

    si g est vide
        graphe_obs    <- SimpleDiGraph(1)
        positions_obs <- [Point2f(0.5, 0.0)]
        couleurs_obs  <- [:bleu]
    finSi
    sinon
        positions_obs <- pos
        couleurs_obs  <- col
        graphe_obs    <- g
    finSinon
```

e. L2F2_Dequantification_App.jl

Rôles	Entrée(s)	Sortie(s)	Dépendances internes	Dépendances externes
-------	-----------	-----------	----------------------	----------------------

Point d'entrée de l'application	n/a	n/a	Interface.jl Dequantification.jl Construction.jl Arbre.jl	n/a
---------------------------------	-----	-----	--	-----

Inclut tous les modules dans l'ordre et finit par appeler `créer_interface()` du module `Interface`.

A noter : déclarer le module avec `module L2F2_Dequantification_App`

Ses Fonctions

Appelle `créer_interface()` du module `interface.jl`.

`créer_interface()`

5. Planification

Le développement de l'application dure quatre semaines. Il commence fin février et doit se terminer fin mars.

Le module `arbre.jl` est implémenté en premier par Salim ACHAK. Viennent ensuite `construction.jl` et `déquantification.jl` respectivement implémentés par BENALI Abdallah et KANGNI Dina. Enfin, l'équipe se partage le développement de l'interface graphique. La dernière semaine est consacrée à l'intégration de l'application.

6. Annexe

Figure 2 : Diagramme d'activité de l'application L2F2

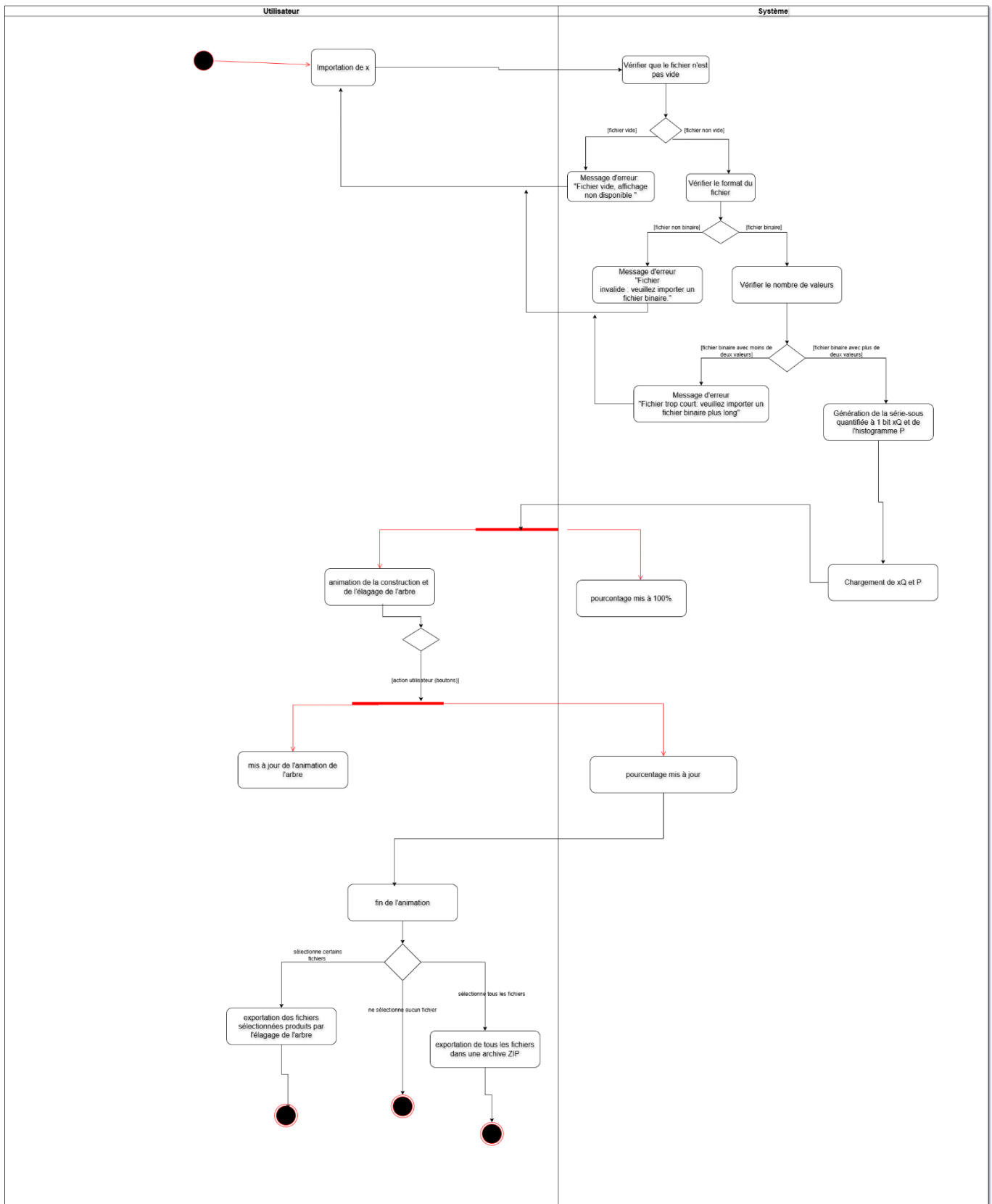


Figure 3 : Architecture complète du projet L2F2

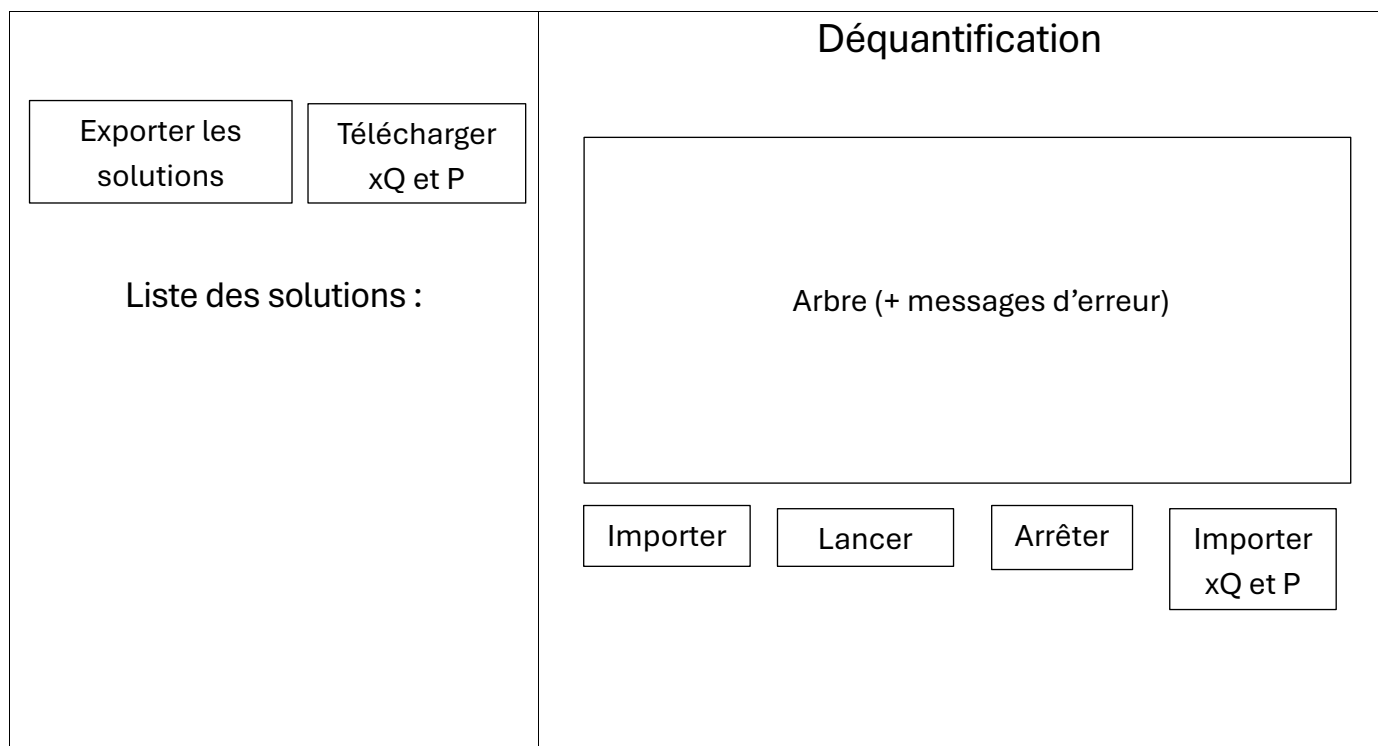
```
.
/  LICENSE.txt
/  Manifest.toml
/  Project.toml
/  README.md
/  run.bat
/  run.sh
+---doc
/      cahier de conception.txt
/      charge.txt
/      plan_test.txt
/      recette.txt
+---src
/  /  application.jl
/  /  interface.jl
/  /
/  +---algortihm
/  /      arbre.jl
/  /      construction.jl
/  /      dequantification.jl
/  /
/  +---data
+---test
/      runtests.jl
/      test_arbre.jl
/      test_construction.jl
/      test_dequantification.jl
```

```

/
+---data
    +---input
        /      P.ppm
        /      x10.dat
        /      x999.dat
        /      x9999.dat
        /      x99994.dat
        /      x_AR1_940.dat
        /      x_AR1_9980.dat
        /      x_AR1_99882.dat
        /
    +---temp

```

Figure 4 : Maquette de l'interface graphique



7. Glossaire

Sous-quantification	<i>Correspond à quantifier une donnée qui a déjà été quantifié</i>
Discrétisé	<i>Fait de remplacer une valeur continue par une valeur discrète</i>
Bit de poids faible	<i>Dernier bit d'un mot binaire. Il contient le moins d'information.</i>
Int16	<i>Entier signé codé sur 16 bits</i>
x	<i>Jeu de données en entrée fourni par l'utilisateur</i>
xQ	<i>X sous quantifié à 1 bit</i>
P	<i>Histogramme de x qui représente le nombre de fois qu'apparaît chaque couple $x(n-1)$ et $x(n)$ dans la série</i>
Observable	<i>Type fourni par la bibliothèque graphique GLMakie dont la modification provoque automatiquement la mise à jour de tous les interface qui l'écotent.</i>

8. Reference

The Julia Language. « Julia 1.12 Documentation ». <https://docs.julialang.org/en/v1.12/>.

KANGNI, D., BENALI, A., & ACHAK, S. (2026). *Cahier des charges L2F2*.

KANGNI, D., ZOUAD.M, (2026). *Cahier de recette L2F2*.

« GLMakie | Makie ». <https://docs.makie.org/stable/explanations/backends/glmakie.html>

« Home · GraphMakie.Jl ». <https://graph.makie.org/>.

GitHub. « NativeFileDialog.Jl/Src/NativeFileDialog.Jl at Master · JuliaGraphics/NativeFileDialog.Jl ».

<https://github.com/JuliaGraphics/NativeFileDialog.jl/blob/master/src/NativeFileDialog.jl>

Segal (dan@seg.al), Dan. « ZipFile.Jl ». Julia Packages.

<https://juliapackages.com/p/zipfile>.

« Observables | Makie ». <https://docs.makie.org/dev/explanations/observables>

« Graph Types · Graphs.Jl ». <https://gdalle.github.io/Graphs.jl/ecosystem/graphtypes/>

9. Index

arbre.jl, 4, 11, 24

Bit de poids faible, 25

Construction.jl, 4, 14, 22

déquantification, 5, 7

dequantification.jl, 4, 9, 17, 20, 24

Discretisé, 25

GLMakie,, 9, 19

GraphMakie,, 9, 19

histogramme *P*, 5, 7, 8

Interface.jl, 4, 19, 22

NativeFileDialog, 9, 19, 26

quantification, 5

sous-quantification, 5

Sous-quantification, 25

xQ, 7, 8, 9, 14, 15, 16, 17, 19, 25

ZipFile, 10, 19, 21, 26